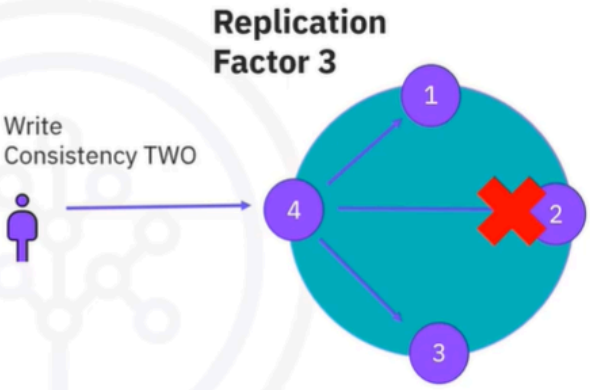
CRUD Operations - Part

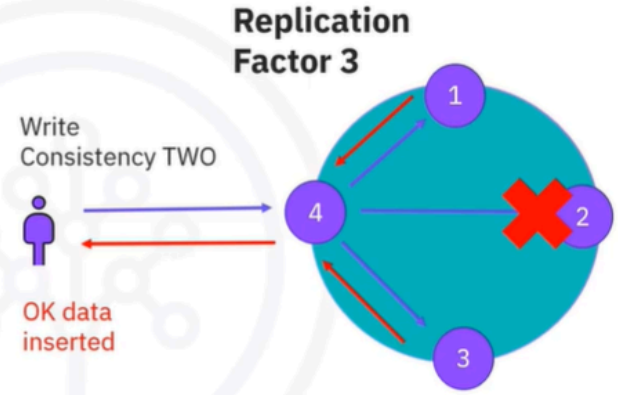
Write operations in Cassandra

- Receiving node is the coordinator for the operation
- Writes directed to all partition replicas
- Acknowledgement expected from consistency number of nodes

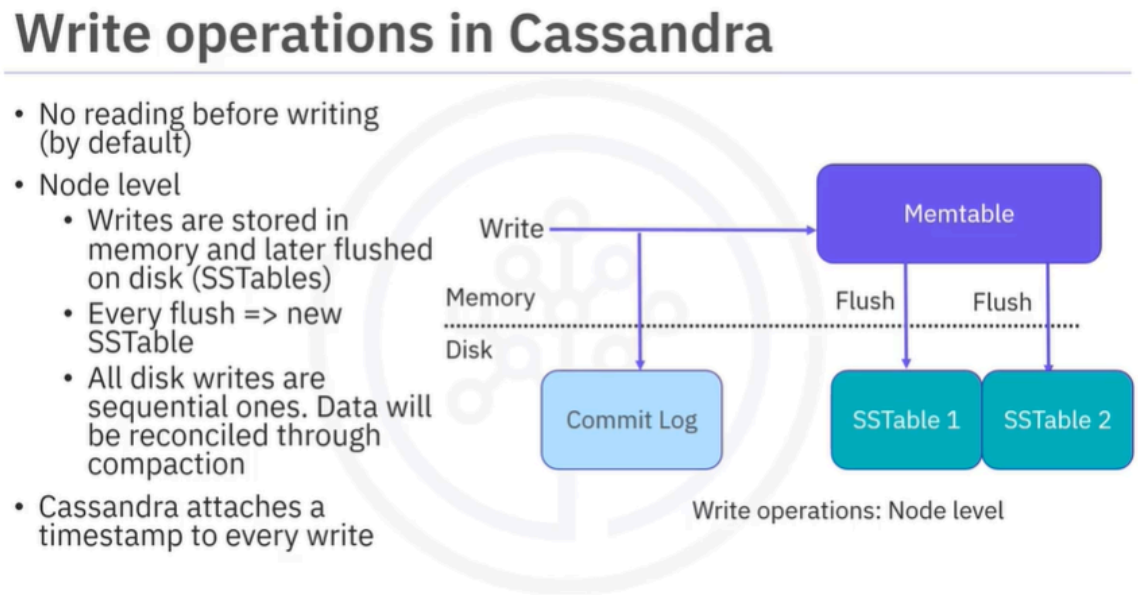


Write operations in Cassandra

- Receiving node is the coordinator for the operation
- Writes directed to all partition replicas
- Acknowledgement expected from consistency number of nodes

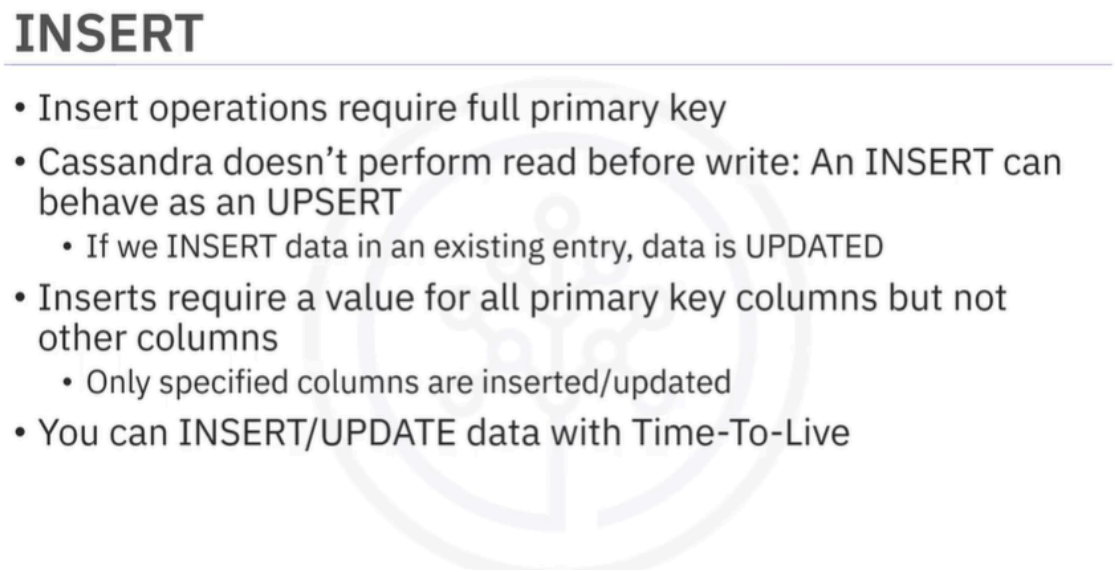


- Before we actually look at the syntax of writes in Cassandra, let's see how a write is done at the cluster and node level. At cluster level, when a write occurs, the node receiving the write becomes the coordinator of the operation. This means that it will make sure to complete the operation and send the result of the write back to the user. Write operations are directed towards all replicas of the partition into which they are writing, but for the operation to be successful, acknowledgement is expected from at least the minimum number of nodes specified for consistency.
- Let's take the example of a keyspace with replication factor 3 and a write operation with consistency set to 2.
- We assume our partition is located in nodes 1, 2, and 3. In this case the write operation arrives in node 4, and node 4 sends the write to nodes 1, 2, and 3 according to the replication factor.
- However, node 2 is not available, so the write acknowledgement is sent only by nodes 1 and 3.Coordinator node 4 checks the number of answers and compares it to the expected consistency value, and then returns an okay.

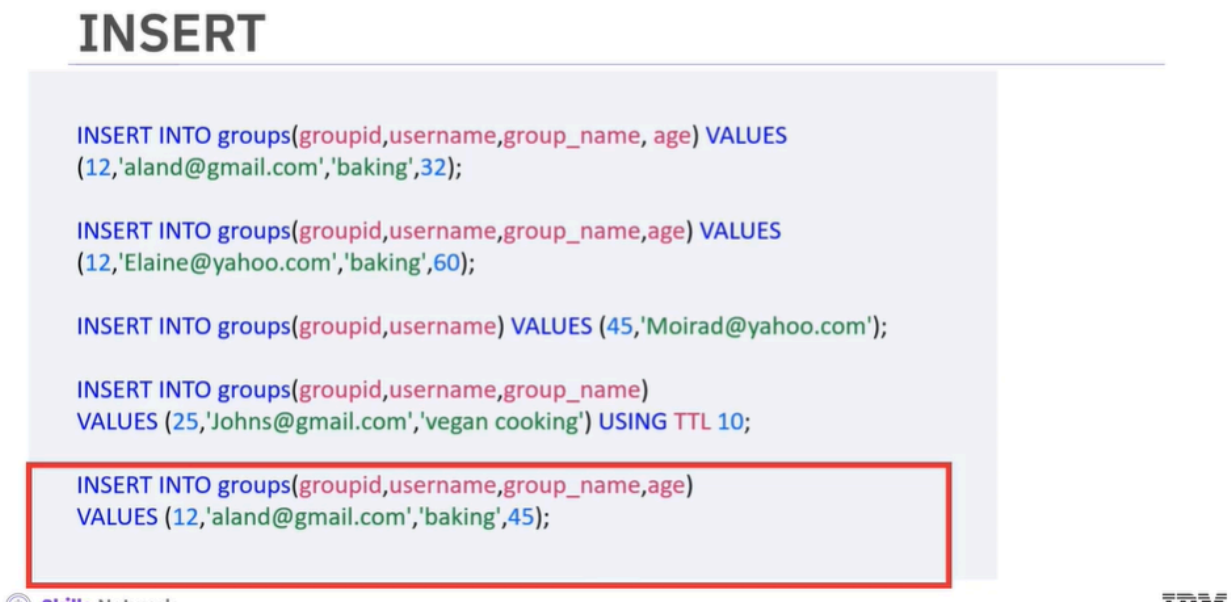


- At the node level there are a few important facts to remember: One important reminder regarding writes in Cassandra: by default, there's no read before the

write operation. At the node level, writes are stored in memory and then flushed on disk in files called SSTables. The more writes we perform, the faster the Memtable gets filled and flushes the data on disk. Every flush operation creates a new file called SSTable. On disk data is appended in subsequent SSTables – later on the data will be optimized on disk through a process called compaction. Cassandra attaches a Timestamp to every write operation. Timestamps are used for data reconciliation. The most recent data wins.



- Let's go through a few basic facts about Inserts in Cassandra. Insert operations require the full primary key to be specified. This means inserts can only be done record by record. Since Cassandra, by default, doesn't perform a read before the write operation, an Insert operation behaves both as an 'Insert' and an 'Upsert' operation.
- If we insert data on an existing entry, then data will be updated. Insert operations require a value for all the primary key columns, but not for the other regular columns specified in the table definition. Only the specified columns will be provisioned with data. You can insert data with a specific Time-To-Live, just like we have done at the table level. Now we can do this at record level.



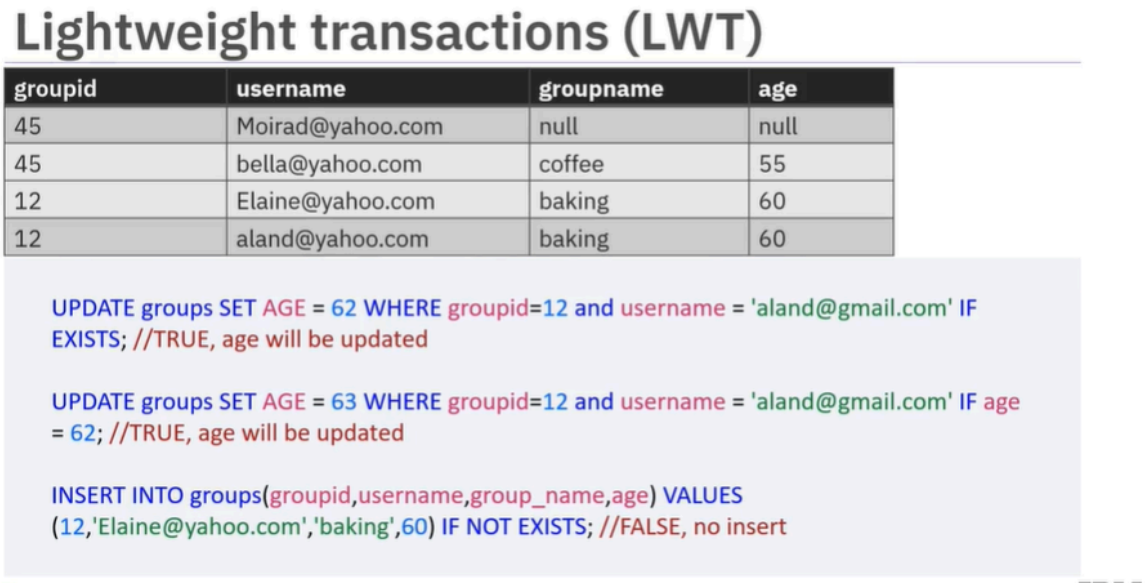
- This means that this data will be visible only for a defined time. Let's see a few examples based on our data. In bold you can see the columns of the primary key: We can insert new data in our table. As you can see, we have specified all the table's columns. In this example we have inserted two users in group 12, the baking group. When inserting data, the Primary key is mandatory. You cannot insert data without fully specifying it.
- As you can see in this example, where we insert a new user in group 45, we only added the mandatory information. Neither the group name nor the user's age had been added. We can also insert data with a Time-To-Live (or TTL). In this case we added a new user to group 25, 'vegan cooking' with a TTL of 10 seconds. This means that in 10 seconds (from insertion) the data will not be available for query anymore. We can use an INSERT as an update when the INSERT is done on existing data. In this case, the age of the user in group 12 will be updated to 45.

UPDATE

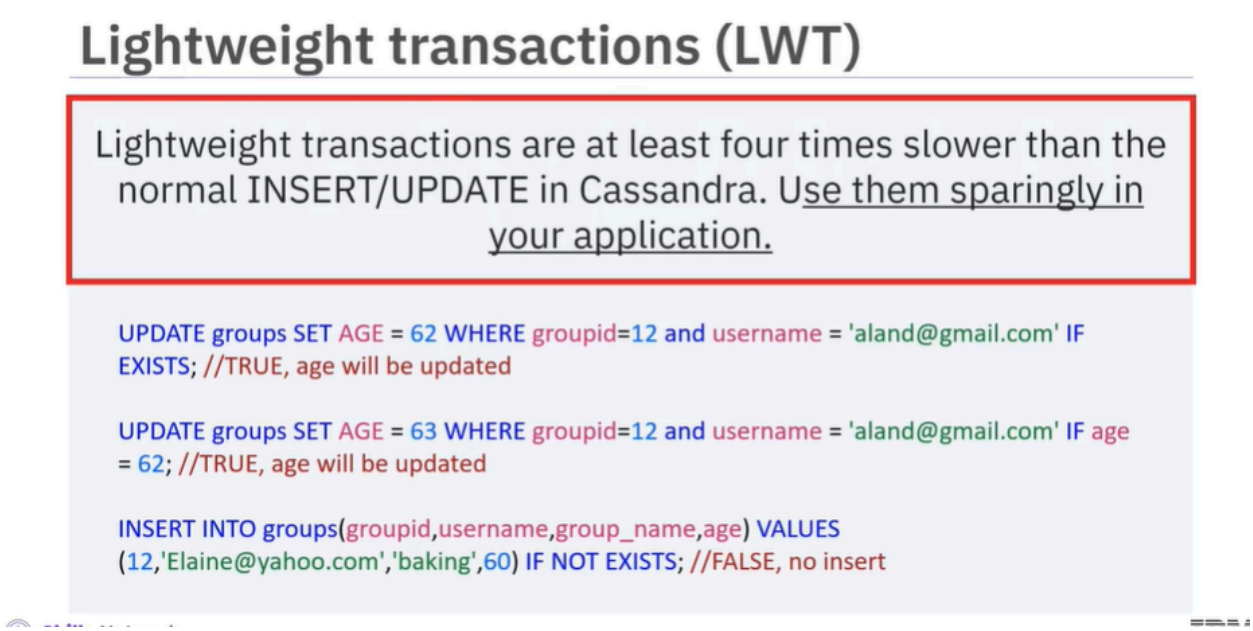
groupid	username	groupname	age
45	Moirad@yahoo.com	null	null
12	Elaine@yahoo.com	baking	62
12	aland@yahoo.com	baking	62

```
UPDATE groups SET group_name = 'grilling' WHERE groupid = 45;

UPDATE groups SET AGE = 60 WHERE groupid = 12 and username ='aland@gmail.com';

UPDATE groups SET age = 55, group_name = 'coffee' WHERE groupid = 20 and username =
'bella@yahoo.com';
```

- IIf we do a ‘Select’ on our table this is what it is going to look like, after all the inserts. This is a cqlsh view of the data, so don't be misled by the ‘null’ value. In this case it means that these cells have no data. Let's do two updates to our data:
- Let's update the name of group 45. Because it is a static column we can update it using only the partition key. This is the only situation in which the UPDATE command does not require a full primary key to be mentioned. We can also update an existing record.
- In this case, we will update the age of a user in group 12. We can call the UPDATE command on a non-existing entry and in this case, UPDATE will behave as an INSERT.



- Let's see what a Select on our table would yield at this moment. As you have seen, INSERT and UPDATE can behave in a fairly similar manner, given that by default, Cassandra doesn't locate and read data before executing a WRITE. However, it is possible to instruct Cassandra to look for the data, read it, and only then perform a given operation. This is possible through Lightweight Transactions. Syntax-wise, Lightweight Transactions are supported by introducing IF in INSERT and UPDATE statements. Let's see some examples:
    - We can update the age of a user in a group only if that record exists. We can update the age of a user in a group only if the record exists and the age has a certain value. We can insert data into a Cassandra table only if that data does not exist.
    - In our case the record exists, and in this case no INSERT will be performed.



- Lightweight transactions are at least four times slower than the normal INSERT/UPDATE in Cassandra. You need to use them sparingly in your application.